

《数据科学》实验报告

班级	003	姓名	史润	学号	20241220
实验题目	机器学习分析方法				
实验时间	2025-11-1	实验地点	DS1408		
学年学期	2025-2026(1)	实验性质	☐ 验证性 ☐ 设计性 ☒ 综合性		

1. 探索性数据分析

加载数据

1) 缺失值分析：编程计算并输出每个特征的缺失值数量及其占总样本的比例。

代码：

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

plt.rcParams["font.sans-serif"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False
# 加载数据并查看基本信息
df = pd.read_csv("transformer_full.csv") # 加载数据集
print("数据集基本信息：")
print(f"样本数量: {df.shape[0]}, 特征数量: {df.shape[1]}")
print("\n 数据类型：")
print(df.dtypes)
print("\n 数据统计描述（数值型特征）：")
print(df.describe().round(2)) # 保留 2 位小数，快速了解数据分布
print()
# -----缺失值分析-----#
# 计算缺失值数量和占比
missing_count = df.isnull().sum() # 每个特征的缺失值数量
missing_ratio = (missing_count / len(df)) * 100 # 缺失值占总样本的比例（%）
# 整理为 DataFrame，按缺失比例降序排列
missing_df = pd.DataFrame(
    {
        "特征名称": missing_count.index,
        "缺失数量": missing_count.values,
        "缺失比例(%)": missing_ratio.round(2),
    }
).sort_values("缺失比例(%)", ascending=False)
print("缺失值分析结果：")
```

```
print(missing_df[missing_df["缺失数量"] > 0]) # 只显示有缺失值的特征
if (missing_count == 0).all():
    print("所有特征均无缺失值")
```

结果:

缺失值分析结果:			
	特征名称	缺失数量	缺失比例(%)
油温	油温	27	5.29
油中气体含量	油中气体含量	27	5.29
绕组温度	绕组温度	26	5.10
局部放电量	局部放电量	26	5.10
噪声水平	噪声水平	26	5.10
绝缘电阻	绝缘电阻	25	4.90
振动幅度	振动幅度	25	4.90
负载率	负载率	25	4.90
温度差值	温度差值	25	4.90
局部放电_标准分	局部放电_标准分	25	4.90

2) 异常值分析: 基于属性的物理意义与统计方法 (如 IQR 方法或 3σ 原则), 识别并统计每个特征的异常值数量。

代码:

```
# -----异常值识别与统计
def detect_outliers(feature_series, feature_name):
    data = feature_series.dropna()
    if data.empty:
        return pd.Series(dtype="float64")
    if feature_name in features_normal_range: # 基于元数据的物理范围异常
        low, high = features_normal_range[feature_name]
        phys_outliers = data[(data < low) | (data > high)]
    else:
        phys_outliers = pd.Series() # 无元数据时, 跳过物理范围过滤
    remaining_data = data.drop(phys_outliers.index) if not phys_outliers.empty
    if len(remaining_data) < 4:
        iqr_outliers = pd.Series() # 数据量过少, 无法计算 IQR
    else:
        Q1 = remaining_data.quantile(0.25)
        Q3 = remaining_data.quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        iqr_outliers_data = remaining_data[
            (remaining_data < lower_bound) | (remaining_data > upper_bound)
        ]
    pieces = [s for s in (phys_outliers, iqr_outliers_data) if not s.empty]
    return (
        pd.concat(pieces).drop_duplicates() if pieces else pd.Series()
    ) # 合并异常值 (去重)
outlier_stats = {} # 统计每个特征的异常值
for feature in df.columns:
```

```

outliers = detect_outliers(df[feature], feature)
valid = len(df[feature].dropna())
outlier_stats[feature] = {
    "异常值数量": len(outliers),
    "异常值占有效样本比例(%)": (
        round(len(outliers) / valid * 100, 2) if valid else 0.0
    ),
}
outlier_df = pd.DataFrame(outlier_stats).T.sort_values("异常值数量", ascending=False)
print("异常值分析结果（结合属性的物理意义和 IQR 方法）：")
print(outlier_df[outlier_df["异常值数量"] > 0]) # 只显示有异常值的特征
if (outlier_df["异常值数量"] == 0).all():
    print("所有特征均无异常值")

```

结果：

异常值分析结果（结合属性的物理意义和IQR方法）：		
	异常值数量	异常值占有效样本比例(%)
温度差值	11.0	2.27
油温	9.0	1.86
噪声水平	6.0	1.24
局部放电量	6.0	1.24
局部放电_标准分	5.0	1.03
油中气体含量	4.0	0.83
负载率	4.0	0.82
振动幅度	3.0	0.62
绕组温度	2.0	0.41

3) 属性冗余分析：编程计算特征间的协方差矩阵和相关系数矩阵；绘制相关系数矩阵（热力图），识别可能存在冗余的高相关特征对（ $|r| > 0.8$ ），思考该特征对是否属于冗余。

```

# -----属性冗余分析-----#
cov_matrix = df.cov().round(2) # 计算协方差矩阵
corr_matrix = df.corr(method="pearson").round(3) # 计算 Pearson 相关系数矩阵
# 输出协方差矩阵
print("协方差矩阵：")
print(cov_matrix)
print()

```

结果：

协方差矩阵：	绕组温度	油温	油中气体含量	绝缘电阻	局部放电量	振动幅度	噪声水平	负载率	温度差值	局部放电_标准分
绕组温度	241.88	19.58	7.79	-7.283492e+05	22.46	-1.58	21.42	67.41	33.34	0.66
油温	19.58	55.56	0.31	-2.424472e+06	5.06	4.30	2.81	13.76	-34.77	0.14
油中气体含量	7.79	0.31	102.99	-9.266293e+07	69.28	-9.65	-2.86	5.59	3.85	7.56
绝缘电阻	-7283491.67	-2424471.70	-92662927.25	1.227602e+14	-98720800.17	4454196.02	85106.04	-4618567.83	-374517.64	-8454964.60
局部放电量	22.46	5.06	69.28	-9.072080e+07	493.16	4.39	-0.44	9.32	-0.78	9.32
振动幅度	-1.58	4.30	-9.65	4.454196e+06	4.39	196.25	37.84	-5.95	-11.79	-0.50
噪声水平	21.42	2.81	-2.86	8.510604e+04	-0.44	37.84	36.15	25.57	15.29	-0.39
负载率	67.41	13.76	5.59	-4.618568e+06	9.32	-5.95	25.57	91.36	50.25	0.46
温度差值	33.34	-34.77	3.85	-3.745176e+05	-0.78	-11.79	15.29	50.25	67.76	0.19
局部放电_标准分	0.66	0.14	7.56	-8.454965e+06	9.32	-0.50	-0.39	0.46	0.19	1.00

```

plt.figure(figsize=(12, 10)) # 绘制相关系数矩阵热力图
sns.heatmap(
    corr_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".3f",

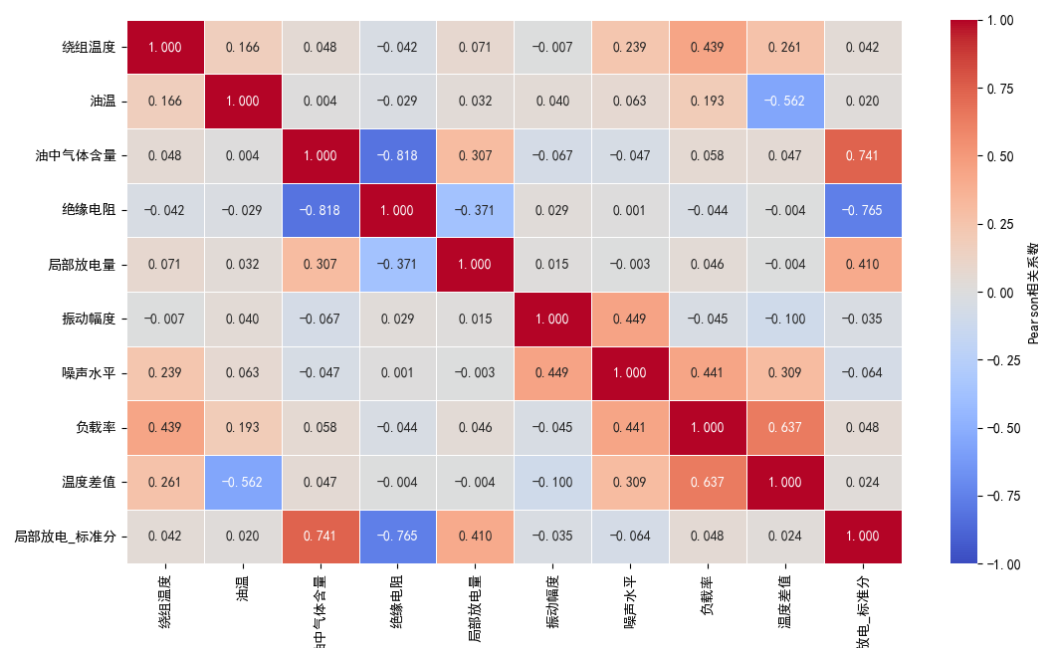
```

```

linewidths=0.5,
vmin=-1,
vmax=1,
cbar_kws={"label": "Pearson 相关系数"},
) # 绘制热力图, annot=True 显示相关系数数值, cmap='coolwarm' 区分正负相关
plt.title("特征相关系数矩阵热力图", fontsize=16, pad=20)
plt.tight_layout()
plt.savefig("correlation_heatmap.png", dpi=300, bbox_inches="tight")
plt.show()
结果:

```

特征相关系数矩阵热力图



```

high_corr_pairs = [] # 识别高相关特征对 (|r|>0.8)
features = corr_matrix.columns
# 遍历上三角矩阵 (避免重复配对, 如(A,B)和(B,A))
for i in range(len(features)):
    for j in range(i + 1, len(features)):
        corr_val = corr_matrix.iloc[i, j]
        if abs(corr_val) > 0.8:
            high_corr_pairs.append(
                {
                    "特征 1": features[i],
                    "特征 2": features[j],
                    "相关系数": corr_val,
                    "相关类型": "正相关" if corr_val > 0 else "负相关",
                }
            )
print("高相关特征对 (|r|>0.8): ") # 输出高相关特征对
if high_corr_pairs:

```

```
high_corr_df = pd.DataFrame(high_corr_pairs)
print(high_corr_df)
else:
    print("无高相关特征对")
```

结果:

```
高相关特征对 (|r|>0.8) :
      特征1  特征2  相关系数  相关类型
0  油中气体含量  绝缘电阻  -0.818  负相关
```

2. 数据清洗

1) 元组冗余处理：删除所有完全重复的行。

```
print("2.数据清洗")
# 2-1 元组冗余处理：删除完全重复行
dup_cnt = df.duplicated().sum()
df = df.drop_duplicates()
print(f"删除完全重复行: {dup_cnt} 条")
print()
```

结果:

```
2.数据清洗
删除完全重复行: 10 条
```

2) 处理缺失值：使用合适的策略（如均值/中位数填充等）填充所有缺失值，并简要说明所选策略的理由。

```
# 2-2 缺失值处理
# 策略说明:
# - 数值型特征：用中位数填充，避免均值受极端值影响
# - 类别型特征：用众数填充，保证出现频率最高的类别
num_cols = df.select_dtypes(include=np.number).columns
cat_cols = df.select_dtypes(exclude=np.number).columns
for col in num_cols:
    if df[col].isnull().any():
        median_val = df[col].median()
        df[col] = df[col].fillna(median_val)
        print(f"【{col}】缺失值用中位数 {median_val} 填充")
print()
for col in cat_cols:
    if df[col].isnull().any():
        mode_val = df[col].mode()[0]
        df[col] = df[col].fillna(mode_val)
        print(f"【{col}】缺失值用众数 '{mode_val}' 填充")
print()
```

结果:

```
【绕组温度】缺失值用中位数 62.38903970406037 填充
【油温】缺失值用中位数 59.20691052379994 填充
【油中气体含量】缺失值用中位数 50.09437740692551 填充
【绝缘电阻】缺失值用中位数 71966461.28733094 填充
【局部放电量】缺失值用中位数 -14.13630328107618 填充
【振动幅度】缺失值用中位数 64.5544188639726 填充
【噪声水平】缺失值用中位数 66.35270156106178 填充
【负载率】缺失值用中位数 62.40626674764414 填充
【温度差值】缺失值用中位数 2.874970714744478 填充
【局部放电_标准分】缺失值用中位数 0.0046182721688862 填充
```

3) 处理异常值: 使用恰当的策略 (如删除、中位数填充等) 处理异常值。

2-3 异常值处理

策略说明:

- 将 detect_outliers 识别出的异常值用该特征的中位数替换

- 既保留样本其余信息, 又消除极端影响

outlier_cnt = 0

for col in df.columns:

 outliers = detect_outliers(df[col], col)

 if not outliers.empty:

 median_val = df[col].median()

 df.loc[outliers.index, col] = median_val

 outlier_cnt += len(outliers)

 print(f"【{col}】{len(outliers)} 个异常值已替换为中位数 {median_val}")

print(f"异常值处理完成, 共处理 {outlier_cnt} 个异常样本")

结果:

```
【绕组温度】2 个异常值已替换为中位数 62.38903970406037
【油温】10 个异常值已替换为中位数 59.20691052379994
【油中气体含量】6 个异常值已替换为中位数 50.09437740692551
【绝缘电阻】1 个异常值已替换为中位数 71966461.28733094
【局部放电量】7 个异常值已替换为中位数 -14.13630328107618
【振动幅度】4 个异常值已替换为中位数 64.5544188639726
【噪声水平】14 个异常值已替换为中位数 66.35270156106178
【负载率】4 个异常值已替换为中位数 62.40626674764414
【温度差值】16 个异常值已替换为中位数 2.874970714744478
【局部放电_标准分】11 个异常值已替换为中位数 0.0046182721688862
```

异常值处理完成, 共处理 75 个异常样本

3. 数据采样 (简单随机抽样对比)

1) 从清洗后的数据集中, 有放回地随机抽取 100 条样本, 计算“绕组温度”的均值。

2) 无放回地随机抽取 100 条样本, 同样计算“绕组温度”的均值。

3) 重复上述过程 100 次, 绘制两个均值分布的直方图, 对比两种抽样方式的方差 (均值分布特征) 差异。

代码:

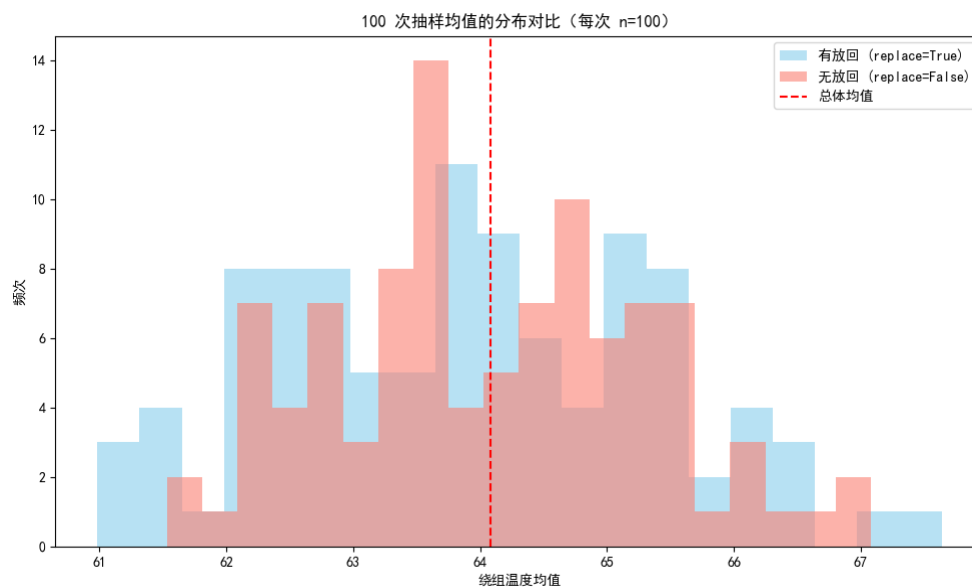
print("3. 数据采样 (简单随机抽样对比)")

```
target_col = "绕组温度"
if target_col not in df.columns:
    print("未找到字段 '{}', 请检查列名!".format(target_col))
else:
    n_sample = 100 # 每次抽 100 条
    n_repeat = 100 # 重复 100 次
    mean_replace = [] # 有放回均值列表
    mean_no_replace = [] # 无放回均值列表

    for _ in range(n_repeat):
        # 有放回
        sample_r = df[target_col].sample(n=n_sample, replace=True)
        mean_replace.append(sample_r.mean())
        # 无放回
        sample_nr = df[target_col].sample(n=n_sample, replace=False)
        mean_no_replace.append(sample_nr.mean())
    # 画直方图
    plt.figure(figsize=(10, 6))
    plt.hist(
        mean_replace, bins=20, alpha=0.6, label="有放
回 (replace=True)", color="skyblue"
    )
    plt.hist(
        mean_no_replace,
        bins=20,
        alpha=0.6,
        label="无放回 (replace=False)",
        color="salmon",
    )
    plt.axvline(df[target_col].mean(), color="red", linestyle="--", label="
总体均值")
    plt.xlabel("绕组温度均值")
    plt.ylabel("频次")
    plt.title(f"{n_repeat} 次抽样均值的分布对比 (每次 n={n_sample})")
    plt.legend()
    plt.tight_layout()
    plt.savefig("sampling_compare.png", dpi=300)
    plt.show()

    var_r = np.var(mean_replace, ddof=1)
    var_nr = np.var(mean_no_replace, ddof=1)
    print(f"有放回    均值分布方差: {var_r:.6f}")
    print(f"无放回    均值分布方差: {var_nr:.6f}")
    print(f"方差减少率: {(var_r - var_nr) / var_r * 100:.2f}%")
```

结果:



```
3. 数据采样 (简单随机抽样对比)
有放回  均值分布方差: 2.178206
无放回  均值分布方差: 1.560669
方差减少率: 28.35%
```

4. 特征离散化方法

1) 对“负载率”这一连续特征，分别使用以下三种方法将其离散化为 3 个类别：

等宽离散化 (提示: `pd.cut`)

等频离散化 (提示: `pd.qcut`)

基于业务规则的离散化 (如: 低负载 [0 - 40%]、中负载 [40 - 70%]、高负载 [70 - 100%])

2) 统计每种方法下各类别的样本数量，并讨论不同方法在反映业务上的差异。

代码:

```
print("4. 特征离散化方法")
discrete_col = "负载率(%)"
if discrete_col not in df.columns:
    # 如果列名没有括号，再试一次
    discrete_col = "负载率"
if discrete_col not in df.columns:
    print("未找到字段 '{}', 请检查列名!".format(discrete_col))
else:
    # 1) 负载率离散化
    # ---- 等宽离散化 ----
    width_labels = ["低负载", "中负载", "高负载"]
    df["等宽离散"] = pd.cut(df[discrete_col], bins=3, labels=width_labels)
    # ---- 等频离散化 ----
```

```
freq_labels = ["低频", "中频", "高频"]
df["等频离散"] = pd.qcut(df[discrete_col], q=3, labels=freq_labels)

# ---- 业务规则离散化 ----
def business_rule(x):
    if x < 40:
        return "低负载"
    elif x < 70:
        return "中负载"
    else:
        return "高负载"

df["业务规则离散"] = df[discrete_col].map(business_rule)

# 2) 统计各类别样本数
count_width = df["等宽离散"].value_counts().sort_index()
count_freq = df["等频离散"].value_counts().sort_index()
count_biz = df["业务规则离散"].value_counts().sort_index()
print("等宽离散化样本分布: ")
print(count_width)
print("\n 等频离散化样本分布: ")
print(count_freq)
print("\n 业务规则离散化样本分布: ")
print(count_biz)

# 3) 可视化对比
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
count_width.plot(kind="bar", ax=axes[0], color="skyblue", title="等宽离散")
count_freq.plot(kind="bar", ax=axes[1], color="orange", title="等频离散")
count_biz.plot(kind="bar", ax=axes[2], color="green", title="业务规则离散")

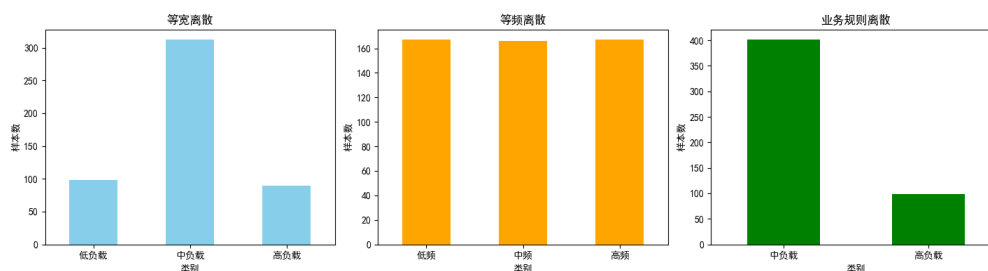
for ax in axes:
    ax.set_ylabel("样本数")
    ax.set_xlabel("类别")
    ax.tick_params(axis="x", rotation=0)
plt.tight_layout()
plt.savefig("discrete_compare.png", dpi=300)
plt.show()

# 4) 简单业务讨论
print("\n=== 业务差异讨论 ===")
print("1. 等宽离散: 区间等宽, 但样本数可能极不均衡; "
```

"当数据分布偏斜时，高负载段样本极少，不利于建模。"

```
)
print(
    "2. 等频离散：每段样本数相等，统计稳定性好，"
    "但可能把相近负载值分到不同段，业务解释性弱。"
)
print(
    "3. 业务规则离散：按实际运行经验划分，解释性最强，"
    "且能与运维手册直接对应；缺点是需人工设定阈值。"
)
)
```

结果：



等宽离散化样本分布：

等宽离散

低负载 98

中负载 312

高负载 90

Name: count, dtype: int64

业务规则离散化样本分布：

业务规则离散

中负载 401

高负载 99

Name: count, dtype: int64

等频离散化样本分布：

等频离散

低频 167

中频 166

高频 167

Name: count, dtype: int64

=== 业务差异讨论 ===

1. 等宽离散：区间等宽，但样本数可能极不均衡；当数据分布偏斜时，高负载段样本极少，不利于建模。
2. 等频离散：每段样本数相等，统计稳定性好，但可能把相近负载值分到不同段，业务解释性弱。
3. 业务规则离散：按实际运行经验划分，解释性最强，且能与运维手册直接对应；缺点是需人工设定阈值。

5. 标准化处理方法

编写代码分别实现 min-max 归一化、Z-score 标准化和小数定标的标准化方法（要求封装为自定义函数）

代码：

```
print("5. 标准化处理方法")

def min_max_scale(series): # min-max 归一化函数封装
    """Min-Max 归一化 -> [0, 1]"""
    return (series - series.min()) / (series.max() - series.min())

def z_score_scale(series): # Z-score 标准化函数封装
    """Z-score 标准化 -> 均值 0, 方差 1"""
    return (series - series.mean()) / series.std()

def decimal_scaling(series): # 小数定标的标准化函数封装
    """小数定标标准化 -> 除以 10^k, 使绝对值最大 < 1"""
    k = int(np.ceil(np.log10(series.abs().max())))
    return series / (10**k)
```

对数值型字段做演示（取前 5 列示意）

```
demo_cols = df.select_dtypes(include=np.number).columns[:5]
```

```
scaled_df = df[demo_cols].copy()
```

```
for col in demo_cols:
```

```
    scaled_df[f"{col}_minmax"] = min_max_scale(df[col])
```

```
    scaled_df[f"{col}_zscore"] = z_score_scale(df[col])
```

```
    scaled_df[f"{col}_decimal"] = decimal_scaling(df[col])
```

打印前 5 行对比

```
print("原始值 vs 三种标准化（前 5 行示例）：")
```

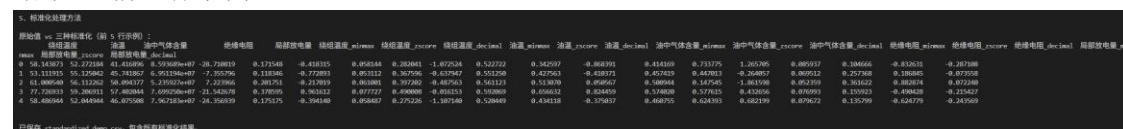
```
print(scaled_df.head())
```

保存完整结果

```
scaled_df.to_csv("standardized_demo.csv", index=False, float_format="%.6f",  
encoding="utf-8-sig")
```

```
print("\n 已保存 standardized_demo.csv，包含所有标准化结果。")
```

结果：（前五行示例）



```
原始值 vs 三种标准化（前 5 行示例）：
原始值 局部放电 油温 油中气体含量 绝缘电阻 局部放电_minmax 局部放电_zscore 局部放电_decimal 油温_minmax 油温_zscore 油温_decimal 油中气体含量_minmax 油中气体含量_zscore 油中气体含量_decimal 绝缘电阻_minmax 绝缘电阻_zscore 绝缘电阻_decimal 局部放电_ps
0 58.143873 52.472484 41.410496 8.503609e+07 28.730819 0.177548 -0.418155 0.055144 0.362061 -1.07524 0.522732 0.342597 -0.889391 0.414105 0.711775 1.365785 0.005017 0.106664 -0.418155 -0.287188
1 51.113193 55.320862 45.341802 9.951394e+07 7.250796 0.134546 -0.270811 0.051123 0.362796 -0.437947 0.515260 0.422593 -0.458371 0.454322 0.444812 -0.264857 0.005522 0.257548 0.136645 -0.072558
2 61.080548 58.112662 58.094177 5.259374e+07 7.233664 0.201751 -0.217819 0.053881 0.337560 -0.887521 0.541121 0.513870 0.006627 0.540864 0.147545 -1.861598 0.052319 0.363223 0.882824 0.072240
3 71.090498 58.260911 57.040844 7.409249e+07 21.540780 0.298095 0.001122 0.077727 0.460086 -0.001241 0.508469 0.006812 0.484455 0.378868 0.375743 0.414906 0.494993 0.259183 -0.494688 -0.214427
4 58.480884 52.444844 46.879588 7.967181e+07 24.569319 0.175175 -0.394148 0.056487 0.275236 1.387548 0.530449 0.434118 0.379437 0.468705 0.454393 0.562199 0.079622 0.195798 -0.424779 -0.243569
```

6. PCA 降维与可视化：

1) 量纲问题诊断：编程计算清洗后数据各特征的方差，并指出量纲差异最大的特征。

2) 对比 PCA 效果：利用任务 5 中编写的 Z-score 标准化函数，对清洗后的数据进行标准化，然后进行 PCA（降至 2D）；同时，对未标准化的清洗后数据直接进行 PCA（降至 2D）；绘制两者的 2D 散点图进行对比，并分析标准化处理对 PCA 的重要性。

3) 基于标准化后的 PCA 结果，分析第一主成分（PC1）：绘制一个水平条形图（Bar Plot），显示 PC1 对每个原始特征的权重绝对值。分析哪些原始特征对第一主成分的贡献最大？

（提示：pca.components 是一个形状为 (n_components, n_features) 的数组。

pca.components[0, :] 代表第一主成分（PC1）在各个原始特征上的权重（或称“载荷”，Loadings）。载荷反映了原始特征对主成分构成的贡献程度）。

代码：

```
print("6. PCA 降维与可视化")
```

```
from sklearn.decomposition import PCA
```

1) 量纲问题诊断

```
num_df = df.select_dtypes(include=np.number)
```

```
var_series = num_df.var().sort_values(ascending=False)
```

```
print("清洗后数据各特征方差（降序）：")
```

```
print(var_series.round(2))
```

```
max_var_feat = var_series.index[0]
```

```
print()
```

```
print(f"量纲差异最大特征：{max_var_feat}（方差={var_series.iloc[0]:.2f}）")
```

结果：

```
6. PCA 降维与可视化
清洗后数据各特征方差（降序）：
绝缘电阻      1.152816e+14
局部放电量    4.387800e+02
绕组温度      2.014000e+02
振动幅度      1.762400e+02
油中气体含量  8.917000e+01
负载率        7.930000e+01
温度差值      4.862000e+01
油温          4.309000e+01
噪声水平      2.670000e+01
局部放电_标准分 7.800000e-01
dtype: float64

量纲差异最大特征：绝缘电阻（方差=115281592102262.12）
```

2) 对比 PCA 效果

```
pca_2d = PCA(n_components=2, random_state=42)
```

---- 未标准化直接 PCA ----

```
X_raw = num_df.values
```

```
pca_raw = pca_2d.fit_transform(X_raw)
```

---- 清洗后的数据在 Z-score 标准化后 PCA ----

```
X_std = num_df.apply(z_score_scale).values
```

```
pca_std = pca_2d.fit_transform(X_std)
```

绘制 2D 散点图对比

```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
```

```
axes[0].scatter(pca_raw[:, 0], pca_raw[:, 1], alpha=0.6, s=15)
```

```
axes[0].set_title("未标准化直接 PCA")
```

```
axes[0].set_xlabel("PC1")
```

```
axes[0].set_ylabel("PC2")
```

```
axes[1].scatter(pca_std[:, 0], pca_std[:, 1], alpha=0.6, s=15, color="orange")
```

```
axes[1].set_title("Z-score 标准化后 PCA")
```

```
axes[1].set_xlabel("PC1")
```

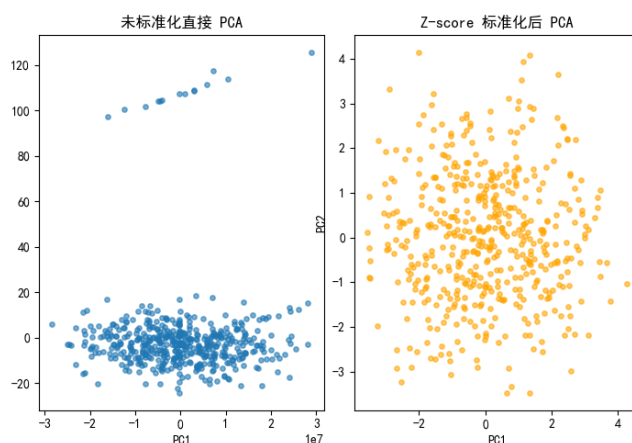
```
axes[1].set_ylabel("PC2")
```

```
plt.tight_layout()
```

```
plt.savefig("pca_2d_compare.png", dpi=300)
```

```
plt.show()
```

结果：



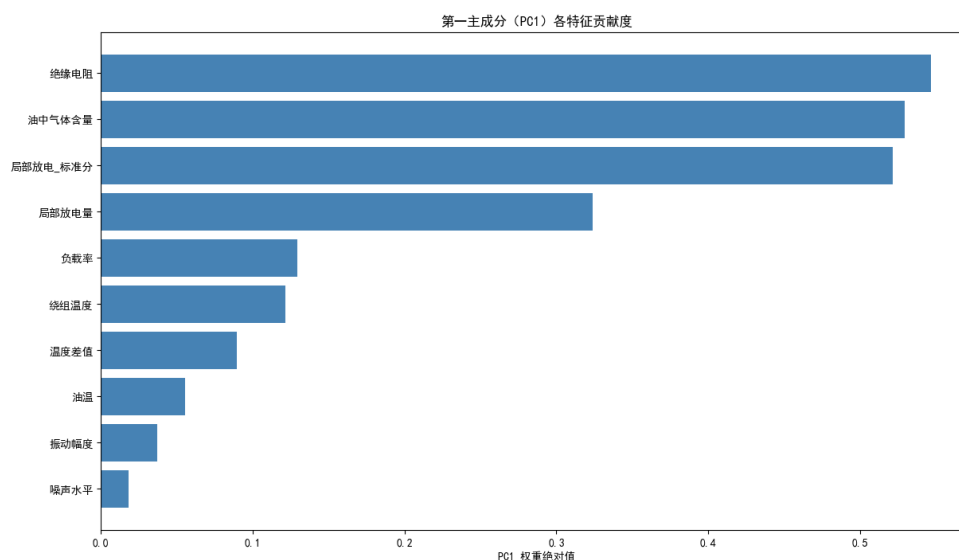
3) 标准化后 PC1 权重分析

```
pca_std_fit = PCA(n_components=2, random_state=42).fit(X_std)
pc1_loadings = pca_std_fit.components_[0] # 第一主成分权重
loadings_abs = np.abs(pc1_loadings)
loadings_df = pd.DataFrame(
    {"特征": num_df.columns, "PC1_权重绝对值": loadings_abs}
).sort_values("PC1_权重绝对值", ascending=True)
```

水平条形图

```
plt.figure(figsize=(6, 8))
plt.barh(loadings_df["特征"], loadings_df["PC1_权重绝对值"], color="steelblue")
plt.xlabel("PC1 权重绝对值")
plt.title("第一主成分 (PC1) 各特征贡献度")
plt.tight_layout()
plt.savefig("pca_pc1_weights.png", dpi=300)
plt.show()
```

结果:



```
print("对 PC1 贡献最大的前 3 个特征: ")
```

```
print(loadings_df.tail(3).iloc[::-1])
```

结果:

```
对 PC1 贡献最大的前 3 个特征:
特征 PC1_权重绝对值
3  绝缘电阻  0.546601
2  油中气体含量  0.529235
9  局部放电_标准分  0.521354
```